

Converting CSS to Qt Style Sheets (QSS) for QtPy6 Applications

Qt Style Sheets (QSS) are similar to CSS but not identical. QSS is essentially a subset of CSS tailored for Qt widgets, with some Qt-specific extensions and **not all CSS features are supported** (Qt's widget style sheets are roughly based on CSS2, lacking many CSS3 properties ¹ ²). Converting a standard CSS stylesheet to QSS requires adjusting unsupported features and translating certain CSS syntax into Qt-compatible forms. Below we outline strategies for conversion – focusing especially on handling **gradients** and **shadows** – and discuss tools, automated approaches, and known limitations.

Differences Between CSS and QSS

Before conversion, it's important to understand key differences:

- **Subset of CSS:** Qt Style Sheets do *not* support many CSS3 properties or selectors. For example, properties like `box-shadow` and `text-shadow` have **no effect in QSS** ² ³. Advanced selectors (like `::before`, `:nth-child`) and layout-related CSS (flex, grid, etc.) are not applicable to QSS, which only styles Qt widgets and sub-controls.
- **Qt-Specific Functions:** QSS introduces its own functions for certain features. Notably, Qt provides `qlineargradient`, `qradialgradient`, and `qconicalgradient` for gradient fills instead of the standard CSS `linear-gradient()` or `radial-gradient()` functions ⁴ ⁵. It also has a `palette()` function to refer to Qt palette colors, etc.
- **Syntax Variations:** Some common CSS syntax must be altered. For instance, Qt expects length units (e.g. "`px`") on numeric values (except a few cases) – a bare number might default to pixels but it's safer to include units ⁶. The RGBA color syntax in Qt historically uses an alpha *percentage* (`rgba(255, 128, 128, 50%)`) whereas standard CSS uses a 0-1 decimal (`rgba(255, 128, 128, 0.5)`), though Qt6 may accept both forms ⁷. These nuances mean a direct copy-paste of CSS may need tweaks to be valid QSS.

Knowing these differences, we can outline how to translate specific features:

Handling Gradient Backgrounds

CSS Gradients (like `linear-gradient(...)` or `radial-gradient(...)`) must be converted to Qt's gradient syntax. Qt supports linear, radial, and conical gradients, but the syntax is different and somewhat more verbose. In QSS you use the functions `qlineargradient()`, `qradialgradient()`, etc., with parameters for geometry and color stops:

- **Linear Gradients:** In CSS you might write, for example:

```
/* CSS example */
background: linear-gradient(to bottom, white 0%, green 100%);
```

In QSS, the equivalent uses `qlineargradient()` with relative coordinates for the gradient vector (from (x1,y1) to (x2,y2) in the widget's bounding box, where `0,0` is top-left and `1,1` bottom-right). The color stops are given with positions 0 to 1:

```
/* QSS example */
QWidget {
    background: qlineargradient(x1:0, y1:0, x2:0, y2:1,
                                stop:0 white, stop:1 green);
}
```

This QSS produces a vertical gradient from white at the top to green at the bottom ⁴. The `x1,y1,x2,y2` define the gradient line; here `x1=0,y1=0` and `x2=0,y2=1` means start at top-center and end at bottom-center (vertical). For a left-to-right gradient, one would use `x1:0, y1:0, x2:1, y2:0` (left to right). Diagonal or angled gradients can be achieved by appropriate coordinate combinations (e.g. `x2:1, y2:1` for top-left to bottom-right ⁴). Qt's gradient coordinates are **normalized (0 to 1)** relative to the widget's area, unlike CSS which often uses degrees or keywords like "to bottom". Thus, an automated converter needs to interpret CSS gradient directions/angles and compute the equivalent `(x1,y1,x2,y2)` coordinates:

- *Example:* `linear-gradient(90deg, red, blue)` (horizontal left-to-right) -> `qlineargradient(x1:0, y1:0.5, x2:1, y2:0.5, stop:0 red, stop:1 blue)`.
- *Example:* `linear-gradient(to bottom right, red, blue)` -> `qlineargradient(x1:0, y1:0, x2:1, y2:1, stop:0 red, stop:1 blue)`.

An automated script could handle common keywords (`to bottom`, `to right`, etc.) or angle values by mapping them to the appropriate normalized vectors.

- **Radial Gradients:** CSS's `radial-gradient(circle at center, ...)` would be written in QSS with `qradialgradient(cx, cy, radius, fx, fy, stops...)`. Qt's radial gradient uses a center (`cx, cy`) and radius (all in relative [0,1] units of the element's bounding box). It also supports a focal point (`fx, fy`) for off-center focal points (for most simple cases `fx, fy` can equal the center). For example, a simple centered radial gradient:

```
/* QSS radial gradient example */
QWidget {
    background: qradialgradient(cx:0.5, cy:0.5, radius:0.5,
                                stop:0 white, stop:1 green);
}
```

This defines a radial gradient centered in the widget (0.5,0.5) with radius covering half the widget (0.5) – effectively a circle that reaches the edges if the widget is a square. The color transitions from white at the

center (stop 0) to green at the perimeter (stop 1) ⁵. In CSS you might have written something like `radial-gradient(circle, white 0%, green 100%)`.

Note: Qt's radial gradients are usually circular. If the widget is not square or you need an elliptical gradient, QSS alone can't specify different x/y radii (you might achieve an ellipse by stretching the widget or using an image). CSS allows specifying `ellipse` and explicit radii or closest-corner, etc., which have no direct analog in QSS. A conversion tool should choose the closest approximation or leave a note for manual adjustment in such cases.

- **Conical (Angular) Gradients:** Qt also supports `qconicalgradient(cx, cy, angle, ...)` for conical gradients (rotational color wheel style) ⁸. Standard CSS now has `conic-gradient()`, but if your CSS uses it, you'd convert similarly. This is less common, but worth mentioning that QSS can do it via `qconicalgradient`.

When converting, remember to sort gradient color stops in ascending order by position (Qt requires stops in order ⁹). An **automated conversion script** can parse `linear-gradient()` or `radial-gradient()` functions from CSS and output the corresponding `qlineargradient` / `qradialgradient` with the computed coordinates and stops. This likely involves parsing the function arguments (which could be done with regex or a CSS parsing library) and handling special cases (like CSS color stop syntax `red 20%` vs QSS `stop:0.2 red`).

Example Conversion (Linear Gradient) – CSS to QSS:

```
/* Original CSS */
.myWidget {
    background: linear-gradient(to right, #ff0000 0%, #0000ff 100%);
}

/* Converted QSS */
QWidget#myWidget {
    background: qlineargradient(x1:0, y1:0.5, x2:1, y2:0.5,
                                stop:0 #ff0000, stop:1 #0000ff);
}
```

In this example, `.myWidget` was presumably a CSS class on an element; in Qt we target it by the widget's name or `objectName` (if the widget has `setObjectName("myWidget")`) or use a Qt-specific "class" selector if set. The gradient goes left-to-right (horizontally) as indicated by the coordinates.

Handling Shadows (Box-Shadow and Text-Shadow)

CSS3 introduced `box-shadow` (for element drop shadows or outlines) and `text-shadow` (for text glyph shadows). **Qt's stylesheet system does** not support these properties** – any `box-shadow` or `text-`

`shadow` declarations will simply be ignored by QSS ² ³. Therefore, converting a CSS design that relies on shadows requires using Qt alternatives or workarounds:

- **Drop Shadows for Widgets:** To achieve a shadow effect around a Qt widget (equivalent to CSS `box-shadow`), you have a few options:
- **Use `QGraphicsDropShadowEffect`:** This is a Qt class that can apply a drop shadow effect to a widget. It's not done via style sheets but via code. For example, in a PyQt/PySide (QtPy) application, you can do:

```
from PyQt6.QtWidgets import QGraphicsDropShadowEffect

button = QPushButton("OK")
shadow = QGraphicsDropShadowEffect()
shadow.setBlurRadius(8)
shadow.setOffset(0, 4)           # 4px drop below
shadow.setColor(QColor(0, 0, 0, 120)) # semi-transparent black
button.setGraphicsEffect(shadow)
```

This approach gives a real shadow under the widget. You can toggle the effect on hover or other events in code if needed (to emulate a hover shadow). While this isn't an automated *stylesheet* conversion, it is the recommended Qt solution for dynamic shadows ¹⁰ ¹¹. You might incorporate such code into your application to complement the style sheet, since QSS can't do it alone.

- **Use images or border-image:** If you want a purely QSS solution for a static shadow, one hack is to use a PNG image of a shadow as a border/frame. Qt style sheets support `border-image` which can slice an image and draw it around a widget border. By creating an image (or 9-patch grid) that looks like a shadow (blurred edges), you can simulate a shadow. For example:

```
QWidget#shadowFrame {
    border: 10px transparent;
    border-image: url("shadow_frame.png") 10 repeat;
}
```

Here `shadow_frame.png` would be an image with transparent center and a drop-shadow effect around the edges. This image is stretched around the widget. This approach can emulate a *static* shadow or glow. The downside is you need to create the image assets for different shadow sizes/looks, and it's not as flexible as a true effect. Nonetheless, it's a known workaround when QSS must be used ².

- **Simulate via gradients:** In some simple cases, you can use a gradient in the widget's background or border to create a shadow-like effect (e.g. a subtle dark gradient at the edges). This is quite limited – essentially an *inner shadow* can be faked by a gradient that darkens toward the edges of a widget's background. However, a true outer drop shadow cannot be done with just a gradient on the same element – you'd need a parent container with a slightly larger background. For instance, one trick is

to have a parent `QWidget` with padding and a slightly larger dark background that peeks out as a “shadow” behind the child widget ². These tricks are usually more effort than they’re worth for general conversion, but they exist.

- **Text Shadows:** QSS cannot directly render text shadows or outlines on `QLabel/QPushButton` text. If your CSS used `text-shadow`, the straightforward approach is to drop it in the QSS and use an effect in code or a custom paint. Qt Widgets don’t have a built-in style for text shadows, but you can:
 - Use **`QGraphicsDropShadowEffect`** on the `QLabel` or any widget that renders text. This will also cast a shadow of the widget’s shape/text. It can work for text (the effect will blur and offset the text drawing). Ensure the widget is raised above any background so the shadow shows.
 - For outlines, you might need a more custom approach (e.g. subclassing the widget and painting the text twice with different colors offset, or using `QPainter::drawText` with an outline). There is no one-line QSS alternative. As one Qt developer noted, “Qt doesn’t support CSS3 ... Have you checked `QGraphicsDropShadowEffect` for the label?” ³ – meaning code-based effects are the go-to solution.

Conversion strategy for shadows: An automated stylesheet converter should **recognize and flag** `box-shadow` and `text-shadow` properties, since they cannot be translated into QSS. The tool might drop them and perhaps output a warning or comment for the developer, suggesting to implement a `QGraphicsEffect` or image-based workaround. For example, the converter could output something like:

```
/* box-shadow: 0 0 10px #000; -- unsupported in QSS, consider
QGraphicsDropShadowEffect */
```

This at least informs the user that manual steps are needed for shadows.

Tools and Automation for Conversion

Converting a non-trivial CSS file to QSS can be complex. While there isn’t a magical one-click converter for all CSS (given the feature gaps), there are **open-source tools and strategies** that can help:

- **QtSass (spyder-ide/qtsass)** – “Compile SASS files to Qt stylesheets.” QtSASS is a tool that allows you to use Sass/SCSS (a CSS preprocessor) to write styles, and it handles Qt-specific quirks when generating the final QSS ¹². For example, QtSASS knows that Qt uses `qlineargradient` with a special syntax, and it provides a function or preprocessor to support it. In normal CSS/Sass, the syntax `qlineargradient(x1:0, y1:0, ... stop:0.5 blue, stop:1 green)` would not parse, so QtSASS lets you write a more CSS-standard form (or uses a placeholder function) and then outputs the correct QSS format ¹³. It also fixes the RGBA syntax difference automatically (converting 0.5 to 50% for alpha) ⁷. Using QtSASS, you can maintain a stylesheet in a format very close to standard CSS (even 100% valid SASS in many cases) and compile it to QSS, which is great for an automated workflow. This tool is particularly useful if you are starting with a CSS or Sass theme – you can integrate Qt-specific adjustments in the Sass (like using `qlineargradient` or removing unsupported rules) and then regenerate QSS whenever needed. QtSASS is open source (MIT licensed) and can be installed via pip. It provides both a CLI (`qtsass input.scss -o output.qss`) and Python API for integration ¹⁴.

- **Custom Conversion Script (using a CSS parser):** If you have to convert arbitrary CSS files, writing a small conversion script can automate the bulk of the work. In Python, you could use libraries like `cssutils` or `tinycss2` to parse the CSS into a DOM of rules and properties. Then programmatically adjust the stylesheet:
- Iterate over rules and for each declaration, check if the property or value needs transformation.
- **Gradients:** Detect values containing `linear-gradient` or `radial-gradient`. Parse the contents (e.g. with regex or by looking at function tokens if using a parser library) and construct the equivalent `qlineargradient/qradialgradient` string. Insert that as the new value. (You may need to remove the prefix if the original had browser prefixes like `-webkit-linear-gradient` – QSS of course doesn't use those).
- **Shadows:** Detect `box-shadow` / `text-shadow` properties and remove them or turn them into comments/warnings in the output.
- **Unsupported properties:** Either drop them or log them. QSS only supports specific properties for specific widget types (as documented in Qt's reference). For instance, properties related to layout (e.g. `display`, `margin` on widgets – QSS margin is different concept) or things like `filter`, `transform`, etc., are not applicable. The script could maintain a whitelist of known QSS properties and filter out anything else.
- **Selectors conversion:** If the CSS uses element selectors (like `button { ... }`), you may need to map these to Qt widget selectors (`QPushButton { ... }`). An automated mapping is tricky unless you know the intended Qt widget for each HTML tag. For example, `<button>` in HTML could be a `QPushButton` in Qt, `<input type=text>` might be `QLineEdit`, etc. If your CSS was written for the web, an automated tool might attempt a guess mapping (e.g., map `body` to `QWidget` or the main window, map common tags to closest Qt equivalents). Alternatively, you might need to manually adjust selectors or set object names in Qt that match CSS classes/IDs used. Qt allows styling by object name (`QWidget#name`) or by a special dynamic property "class" that can mimic CSS classes. A comprehensive conversion might include adding `setProperty("class", "yourclassname")` to widgets so that `QWidget[class="yourclassname"]` selectors work. This is beyond simple stylesheet text conversion but worth noting as a strategy for applying web CSS concepts to Qt widgets.
- **QssStylesheetEditor (hustlei/QssStylesheetEditor):** This is a GUI tool for developing Qt style sheets, featuring live preview and even a concept of variables in QSS which can then be exported to pure QSS. While it's not an automatic converter for an existing CSS file, it can be helpful to interactively refine and test the appearance of the stylesheet in Qt. For instance, after an initial automated conversion, you could load the QSS into this editor to tweak colors or fix any rules that didn't translate well. The editor also supports defining custom variables (similar to CSS custom properties or Sass variables) to ease theme adjustments ¹⁵ ¹⁶.
- **Manual and Semi-automated Approach:** In practice, a fully automated conversion might get you 80% of the way, but you should budget some manual effort for the remaining quirks:
- Use Qt's documentation (the **Qt Style Sheets Reference**) as a guide for what's supported. For example, if your CSS used `border-radius` and `border` properties, QSS does support those (e.g. `border-radius` in QSS rounds widget corners). But if your CSS used something like `box-sizing`

or `outline-offset`, those are not in QSS. You may need to remove or find alternatives for such rules.

- Test the QSS on your Qt application and see if any warnings appear in the console (Qt will often warn about invalid properties or selectors in stylesheets). That can quickly highlight which rules were not understood.
- Keep in mind QSS applies to Qt widget states/pseudo-states too. For example, CSS `:hover` on a button can be written as `QPushButton:hover { ... }` in QSS (Qt supports many pseudo-states like `:hover`, `:disabled`, `:checked`, etc.). However, some widgets might not define all states (e.g. QLabel doesn't have a `:hover` state by default ¹⁷). Ensure the state selectors you convert to actually exist for the widget in question.

Known Limitations and Fallbacks

Even with a good conversion, some web CSS features simply don't have a QSS equivalent. Here are a few noteworthy limitations and how to handle them:

- **Drop Shadows (box-shadow):** *No direct QSS support.* Must use graphics effect or images ¹¹. If an exact shadow effect is critical, plan to implement it in C++/Python (with `QGraphicsDropShadowEffect`) rather than via style sheets.
- **Text Shadows (text-shadow):** *No QSS support.* Use `QGraphicsDropShadowEffect` on the label/text widget, or custom painting. No pure QSS fallback.
- **Multiple Backgrounds/Layers:** In CSS you can layer multiple backgrounds (e.g. an image and a gradient). QSS does not support comma-separated multiple `background` images on a single widget. The workaround is to either compose an image offline that includes the desired look, or use multiple nested widgets (for example, a QFrame behind with one background and a QWidget in front with another). This is usually manual; an automated tool would likely ignore additional backgrounds beyond the first.
- **Advanced CSS3 (filters, transforms, animations):** Not available in QSS. For example, `opacity` in CSS can be done by QWidget's `setOpacity` via Qt GraphicsOpacityEffect, but not via QSS. Transforms (rotate, scale) are not in QSS – you'd use Qt's animation or painting routines. Any CSS animations or transitions would be lost; Qt stylesheets are static and have no concept of keyframe animations.
- **Media Queries or Responsive Design:** Not applicable in QSS – there is no mechanism for media queries. Qt does not automatically adapt styles based on window size or DPI via stylesheets (though Qt will scale units like `pt` for DPI, etc., but that's different).
- **Selector Differences:** In HTML/CSS you might style nested elements (`.container .btn { ... }`). In Qt, the styling is more limited by widget hierarchy and type. An equivalent in QSS would be `QWidget#container QPushButton { ... }` to style buttons inside a container widget with `objectName` "container". QSS supports descendant selectors by widget name or type, but not every complex selector (no general sibling combinators, etc.). If your CSS relies on very specific DOM structure, you might have to restructure it for Qt's widget tree. This typically requires manual intervention – a script could convert simple descendant selectors but might not cover all cases.
- **Pseudo-elements (:before, :after):** These have no counterpart in QSS. Qt style sheets can't inject content or draw extra elements like CSS `::before` can. If your CSS used `::before`/`::after` for decorative purposes, you'll need another approach (perhaps using QLabel or painting). A converter would likely drop those rules entirely.

Despite these gaps, **many CSS styles do translate well**: colors, fonts, padding/margins, borders, border-radius, images, basic selectors by widget type or object name, etc., all work in QSS. For gradients and other backgrounds, QSS provides the needed functionality but via different syntax. The biggest omissions are the dynamic effects like shadows and animations.

Example Summary

To summarize with a concrete example, consider a CSS snippet and its QSS translation:

```
/* Original CSS snippet */
.card {
  background: linear-gradient(45deg, #ffffff, #e0e0e0);
  border: 1px solid #999;
  border-radius: 5px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.3);
  color: #333;
  padding: 10px;
}
```

A possible QSS conversion might look like:

```
/* Converted QSS snippet */
QFrame#card {
  /* 45° diagonal gradient from white to light-gray */
  background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                             stop:0 #ffffff, stop:1 #e0e0e0);
  border: 1px solid #999;
  border-radius: 5px;
  /* box-shadow not supported - consider QGraphicsDropShadowEffect */
  color: #333;
  padding: 10px;
}
```

Here we assumed the “card” is a QFrame widget with `objectName="card"`. The gradient and other basics are converted. The `box-shadow` is commented out or noted as unsupported. The rest of the properties (border, radius, text color, padding) are directly usable in QSS (Qt supports them similarly to CSS).

By using the above approaches – leveraging **tools like QtSASS** for automating syntax fixes ¹², and writing **scripts to handle gradients and filter unsupported properties**, you can **comprehensively convert a stylesheet** for use in a PyQt6/PySide6 (QtPy6) application. Just remember that some visual effects will require jumping outside of style sheets into code (Qt’s paint/effect system) due to QSS limitations ¹¹ ³. With careful mapping of CSS features to QSS equivalents or Qt workarounds, you can achieve a very similar look-and-feel in your Qt application as in a web CSS design.

Sources:

- Qt Company – *Qt Style Sheets Reference*: gradient syntax and QSS capabilities ⁴ ⁵
- Qt Forums – Developer discussions on unsupported CSS features (`box-shadow` , `text-shadow`) ² ³
- *QtSASS* Project – documentation on bridging SASS/CSS with Qt's stylesheet quirks ¹² ¹³
- Rekols (Medium) – "*Qt Button box-shadow Property*" (explains using `QGraphicsDropShadowEffect` since Qt doesn't support CSS3 box-shadow) ¹ .

¹ ¹⁰ Qt Button box-shadow Property. Demand | by Rekols | Medium

<https://medium.com/@rekols90/qt-button-box-shadow-property-c47c7bf58721>

² ¹¹ [SOLVED]box-shadow in stylesheet | Qt Forum

<https://forum.qt.io/topic/30856/solved-box-shadow-in-stylesheet>

³ QT Designer Stylesheet Text shadow / outline | Qt Forum

<https://forum.qt.io/topic/58844/qt-designer-stylesheet-text-shadow-outline>

⁴ ⁵ ⁶ ⁸ ⁹ ¹⁷ Qt Style Sheets Reference | Qt Widgets | Qt 6.10.0

<https://doc.qt.io/qt-6/stylesheet-reference.html>

⁷ ¹² ¹³ ¹⁴ GitHub - spyder-ide/qtsass: Compile SASS files to Qt stylesheets

<https://github.com/spyder-ide/qtsass>

¹⁵ ¹⁶ GitHub - hustlei/QssStyleSheetEditor: Editor for qt stylesheet (qss). Real-time preview, and user can define variables in qss.

<https://github.com/hustlei/QssStyleSheetEditor>